

Freeing rustc's Understanding: Enabling Third-Party Rust Tooling

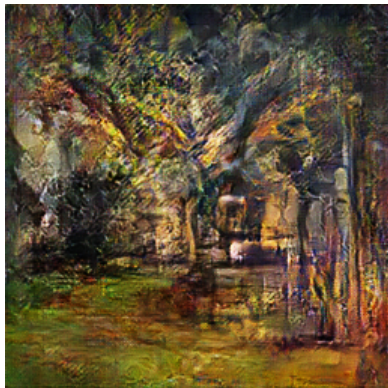
alona.page/talks/freeing-rustcs-understanding.pdf

Alona Enraght-Moony

2026-01-21

whoami (1)

- ▶ Alona Enraght-Moony (she/her)
- ▶ Rustdoc team since 2022
- ▶ Focused there on maintaining rustdoc-json
- ▶ This talk comes from soul-searching about the place that rustdoc-json fits into the ecosystem

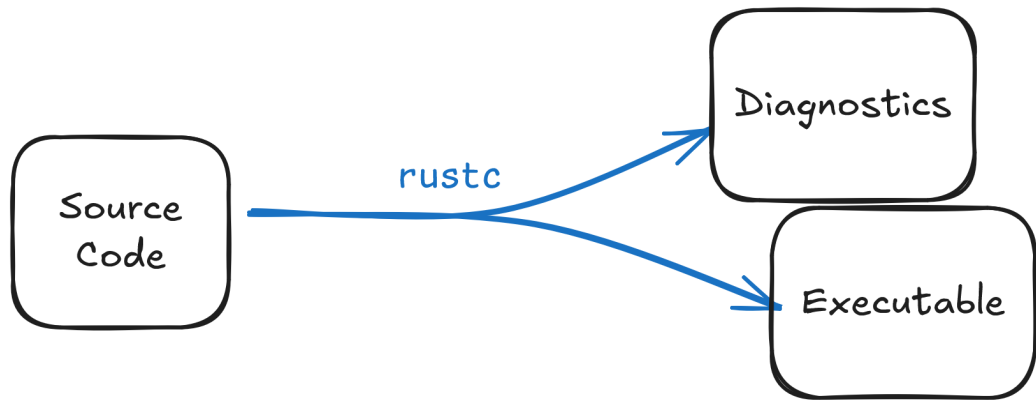


<https://github.com/aDotInTheVoid/>

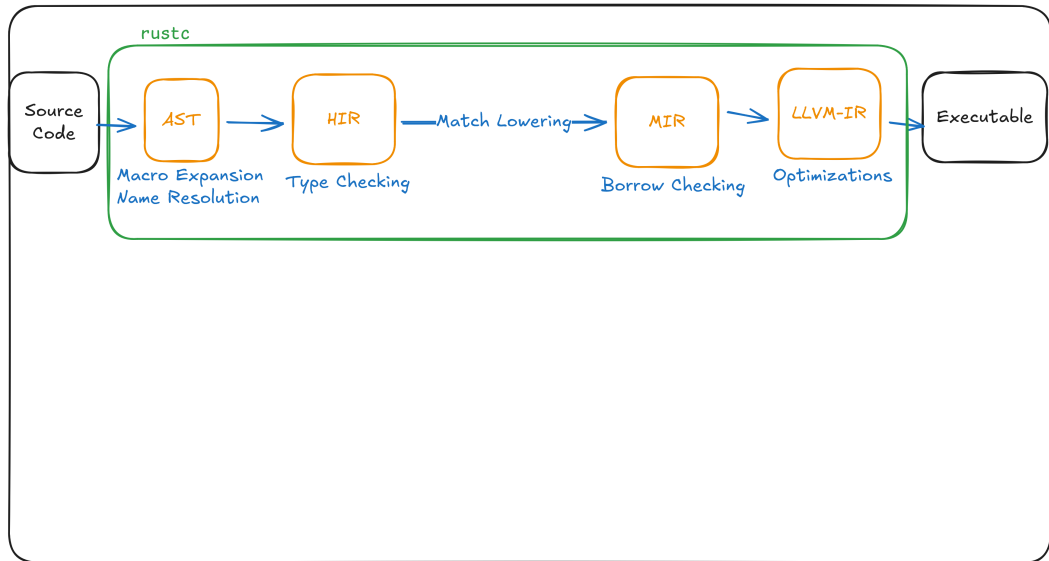
Agenda

- ▶ How the first-party tools work
- ▶ The need for third-party tools
- ▶ Why third-party tooling is much harder than first-party tooling
- ▶ Various approaches to solving this.

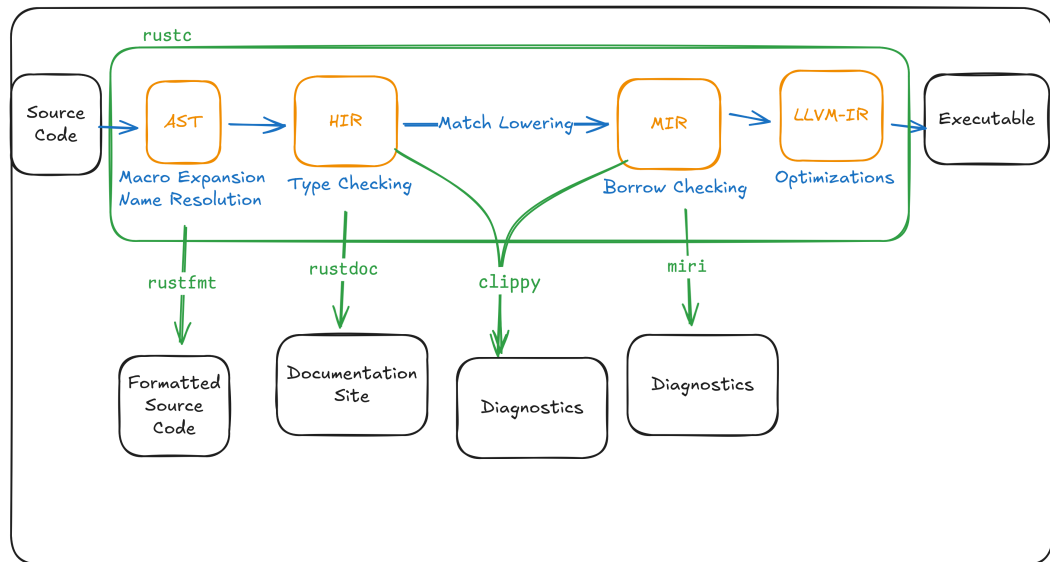
rustc, the 10'000 feet view



rustc, the 1'000 feet view



rustc, and the first party tools



More tools than the rust project can bear to provide

- ▶ Linters
 - ▶ cargo-check-external-types
 - ▶ cargo-semver-checks
 - ▶ klint
 - ▶ dylint
 - ▶ marker
- ▶ Verifiers
 - ▶ kani
 - ▶ aeneas
 - ▶ Creusot
- ▶ Refractoring tools
 - ▶ CoccinelleForRust
- ▶ Inspection
 - ▶ cargo-public-api
 - ▶ cargo-modules

Can third-party be like first party?

```
#![feature(rustc_private)]
extern crate rustc_driver;
extern crate rustc_errors;

// set up target, flags, crate loader into `config`.

rustc_interface::run_compiler(config, |compiler| {
    let ast = rustc_interface::passes::parse(&compiler.sess);
    rustc_interface::create_and_enter_global_ctxt(compiler, ast, |tcx| {
        // real work here.
    })
});
```


Can third-party be like first party?

```
[toolchain]
```

```
channel = "nightly-2025-02-13"
```

```
components = ["llvm-tools-preview", "rustc-dev"]
```

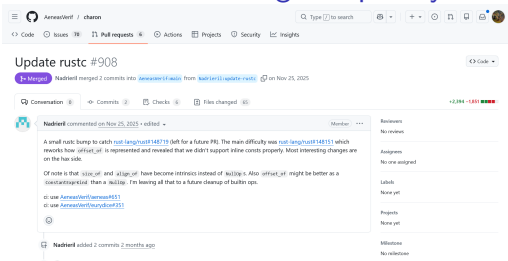
Can third-party be like first party?

```
$ ldd ./target/debug/demo_tool
linux-vdso.so.1 (0x00007ffff7fd6000)
librustc_driver-2fb444505ec98d04.so => /home/alona/.rustup/toolchains/nightly-2026-01-18-x86_64-unknown-linux-gnu/lib/librustc_driver-2fb444505ec98d04.so (0x00007d2cf0a00000)
libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1 (0x00007d2cf86cd000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007d2cf0600000)
libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2 (0x00007d2cf86c8000)
libLLVM.so.21.1-rust-1.94.0-nightly => /home/alona/.rustup/toolchains/nightly-2026-01-18-x86_64-unknown-linux-gnu/lib/../lib/libLLVM.so.21.1-rust-1.94.0-nightly (0x00007d2ce6800000)
librt.so.1 => /lib/x86_64-linux-gnu/librt.so.1 (0x00007d2cf86c1000)
libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0 (0x00007d2cf86bc000)
/lib64/ld-linux-x86-64.so.2 (0x00007d2cf8723000)
libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6 (0x00007d2cf0909000)
libz.so.1 => /lib/x86_64-linux-gnu/libz.so.1 (0x00007d2cf869e000)
```

\$

Problems for third-party tools

rustc internals change frequently



Projects have died due to this:

- ▶ gazebo_lint
- ▶ semverver

Using rustc internals complicates the build

- ▶ rustc can't be build from crates.io with cargo, but needs it's own build system
- ▶ The version of rustc that builds your tool is also the version of rustc your tool uses
- ▶ You need to be on nightly

How the first-party tools avoid this

rustc internals change frequently

Don't allow rustc changes to break first party tools.

Changes to rustc must make sure that rustdoc, clippy, rustfmt, miri tests still pass.

Both of these are possible because first party tools are *developed, tested and distributed* alongside rustc.

Using rustc internals complicates the build

- ▶ First party tools are built with x.py, not cargo
- ▶ First party tools are distributed by rustup, not cargo

```
#![feature(rustc_private)]
```

- ▶ Pros:
 - ▶ Access to all the rich state rustc has
- ▶ Cons:
 - ▶ Only usable on nightly
 - ▶ Large API, constantly evolving
 - ▶ Need to dynamically link to specific version of `librustc_driver-<hash>.so`

rustc_public

- ▶ Pros:
 - ▶ More limited API
- ▶ Cons:
 - ▶ More stable API
 - ▶ Immature, not yet complete
 - ▶ As of today, still need to link to version specific `librustc_driver.so`

compiler-team#949

rust-analyzer

- ▶ Pros:
 - ▶ Published as a library on crates.io
 - ▶ Usable on stable
 - ▶ Entirely freestanding of rustc.
- ▶ Cons:
 - ▶ API changes frequently
 - ▶ Doesn't have a "complete" compiler
 - ▶ No CFG, No Borrow-checking
 - ▶ Optimized for latency, not throughput.

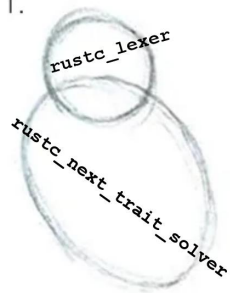
libraryification of rustc

- ▶ `rustc_lexer` and `rustc_next_trait_solver` are on crates.io
- ▶ Pros:
 - ▶ Freestanding
 - ▶ Stable
 - ▶ on crates.io
- ▶ Cons:
 - ▶ Library evolves
 - ▶ Draw the rest of the owl

libraryification of rustc

How to write a rust compiler

1.



2.



1. depend on the librarified bits of rustc

Write 'the rest of the fucking compiler

```
rustdoc --output-format=json
```

- ▶ Pros:
 - ▶ Talks to rustc over a json file, not memory
 - ▶ Your tool can be in stable, but use nightly rustdoc
 - ▶ Can support multiple versions.
 - ▶ Small abi, changes infrequently
- ▶ Cons:
 - ▶ Presentational view, not semantic
 - ▶ Only shows API, nothing about the contents of functions
 - ▶ Can't request the information you need, get all the information whether you care or not

Charon

- ▶ Pros:
 - ▶
- ▶ Cons:
 - ▶ Focused on semantics.
 - ▶ Charon itself requires specific nightly at
 - ▶ Can't request the information you need, get all the information whether you care or not

-Zsave-analysis

This is basically rustdoc-json, but for rustc!

- ▶ Pros:
 - ▶ Consumer code can be enti
- ▶ Cons:
 - ▶ Removed in 2023 (#101841)

Thanks for listening

Thanks

Jynn Nelson, Julia Ryan, David Barsky

Contact

- ▶ me AT alona DOT page
- ▶ fedi: @adotinthevoid@hachyderm.io

Related works

- ▶ <https://matklad.github.io/2023/01/25/next-rust-compiler.html>
- ▶ <https://smallcultfollowing.com/babysteps/blog/2020/04/09/libraryification/>